

NiceVolunteer: Project report

Trabalho realizado por:

Daniel Simbe [110235]

1. Visão Geral do Projeto

1.1 Enquadramento

A plataforma NiceVolunteer responde à necessidade de um sistema centralizado que ligue voluntários a oportunidades no seio da comunidade académica da Universidade de Aveiro. A plataforma promove o envolvimento cívico, reforça os laços comunitários e reconhece as contribuições através de um sistema de pontos.

1.2 Objetivos

Os principais objetivos incluem a criação de um mercado digital que conecte promotores e voluntários, a implementação de um sistema de recompensas baseado em pontos, a garantia de fiabilidade da plataforma através de testes abrangentes, o estabelecimento de um pipeline de integração e entrega contínua, e a disponibilização de mecanismos de monitorização em tempo real.

Os objetivos de qualidade contemplam uma cobertura de testes mínima de 80 por cento, mais de 95 testes automatizados, ausência de vulnerabilidades críticas de segurança, tempos de resposta inferiores a 500 milissegundos para 95 por cento das requisições em carga normal, e taxa de sucesso de builds de 100 por cento.

1.3 Stakeholders

Os voluntários incluem estudantes, docentes, investigadores e funcionários que procuram oportunidades e acumulam pontos. Os promotores correspondem a departamentos, associações estudantis e entidades externas que publicam oportunidades e gerem candidaturas. Os parceiros são organizações que oferecem recompensas. Os administradores são responsáveis pela supervisão da plataforma.

2. Arquitetura do Sistema

A plataforma NiceVolunteer foi concebida com uma arquitetura genérica em camadas, seguindo o modelo cliente-servidor, de forma a garantir modularidade, manutenibilidade e escalabilidade. Esta abordagem permite a separação clara de responsabilidades entre os diferentes componentes do sistema, facilitando a evolução futura e a aplicação de boas práticas de engenharia de software.

O sistema é composto por três grandes blocos funcionais: interface de utilizador, lógica de negócio e persistência de dados, integrados por serviços de monitorização e suporte à qualidade.

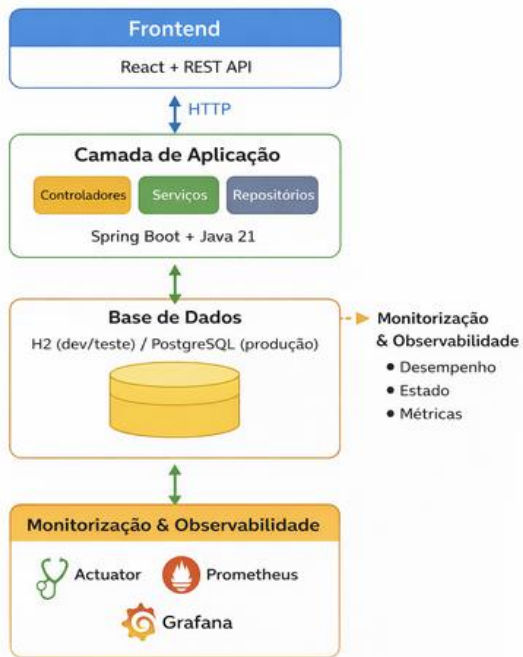
A camada de apresentação corresponde ao frontend web, desenvolvido em React, responsável pela interação com os utilizadores, visualização de oportunidades, submissão de candidaturas e consulta de pontos acumulados. Esta camada comunica exclusivamente através de chamadas REST para o backend.

A camada de aplicação e lógica de negócio é implementada em Spring Boot e concentra toda a funcionalidade do sistema, incluindo autenticação, gestão de voluntários e promotores, processamento de candidaturas, atribuição de pontos e validações. Esta camada organiza-se internamente em controladores que expõem a API, serviços que implementam as regras de negócio e componentes auxiliares de validação e tratamento de exceções.

A camada de persistência é responsável pelo armazenamento e acesso aos dados, utilizando JPA/Hibernate para mapear entidades de domínio para a base de dados relacional. Durante o desenvolvimento e testes é utilizada uma base de dados H2 em memória, sendo o sistema preparado para utilização de PostgreSQL em ambiente de produção.

De forma transversal à arquitetura funcional, encontra-se integrada uma infraestrutura de monitorização e qualidade composta por Spring Boot Actuator, Prometheus e Grafana, que permite recolher métricas de desempenho, estado da aplicação e indicadores de negócio em tempo real.

Esta arquitetura genérica em camadas assegura baixo acoplamento entre componentes, elevada coesão interna e facilidade de teste, sendo particularmente adequada para aplicações web modernas orientadas a serviços.



3. Funcionalidades Implementadas

A plataforma suporta registo de voluntários com validações específicas, autenticação e gestão de sessões, listagem e pesquisa de oportunidades, candidatura a oportunidades com prevenção de duplicados, gestão de candidaturas por promotores, sistema de pontos e histórico de transações, recomendações básicas e monitorização do sistema.

4. Tecnologias Utilizadas

O backend foi desenvolvido com Java, Spring Boot, Hibernate, JPA, H2, PostgreSQL, Actuator e Micrometer.

O frontend utiliza React, Vite, React Router e Axios.

A estratégia de testes recorre a JUnit, Mockito, Spring Boot Test, Cucumber, Cypress, k6 e JaCoCo.

A infraestrutura DevOps inclui Maven, GitHub Actions, Docker, Docker Compose, Prometheus, Grafana, Checkstyle, CodeQL e ESLint.

5. Garantia de Qualidade

A estratégia de qualidade baseou-se em prevenção por análise estática, detecção por testes automatizados e monitorização em tempo real.

Foram definidos quality gates que exigem passagem de todos os testes, cobertura mínima de 80 por cento, ausência de erros de estilo e vulnerabilidades críticas.

Os resultados alcançados incluíram 86 por cento de cobertura de código, 95 testes automatizados, 100 por cento de sucesso nos testes, zero erros de Checkstyle e builds rápidos e estáveis.

6. Estratégia de Testes

Foi implementada a pirâmide de testes com forte incidência em testes unitários, complementados por testes de integração, BDD e end-to-end.

Os testes unitários validaram a lógica de negócio isoladamente, os de integração validaram a comunicação entre componentes, os BDD garantiram requisitos funcionais e os E2E simularam fluxos reais de utilizador.

A cobertura global atingiu 86 por cento em todas as camadas críticas do sistema.

7. Análise de Desempenho

Os testes de carga demonstraram tempos médios de resposta baixos em condições normais.

Os testes de pico mostraram degradação temporária mas recuperação rápida.

Os testes de stress identificaram um limite de cerca de 180 utilizadores concorrentes.

Foram propostas melhorias como caching, otimização da base de dados e escalabilidade horizontal.

8. Pipeline de CI/CD

O pipeline automatiza compilação, execução de testes, verificação de cobertura, análise de estilo, scans de segurança e geração de relatórios.

Apresentou uma taxa de sucesso de 100 por cento e tempos médios de execução inferiores a 30 segundos.

9. Monitorização e Observabilidade

A aplicação expõe métricas técnicas e métricas de negócio recolhidas pelo Prometheus e visualizadas em Grafana.

Os dashboards incluem indicadores de desempenho, estado do sistema, utilização de recursos e métricas funcionais.

Foram configuradas regras de alerta para falhas e degradação de desempenho.

10. Desafios e Soluções

Foram resolvidos problemas de integração frontend-backend através da normalização de APIs, automatização de dados de teste, aumento de cobertura de testes, correção de violações de estilo e documentação de limitações de segurança.

A reorganização da pirâmide de testes reduziu significativamente o tempo de execução.

11. Trabalho Futuro

Inclui melhoria do algoritmo de recomendações, segurança com BCrypt, caching com Redis, escalabilidade horizontal, expansão de funcionalidades frontend, autenticação JWT, sistema de notificações, aplicações móveis, acessibilidade e internacionalização.

12. Conclusão

O projeto cumpre os objetivos funcionais e demonstra práticas sólidas de engenharia de software e garantia de qualidade.

A combinação de testes abrangentes, CI/CD automatizado e monitorização contínua resultou num “sistema fiável, escalável e bem documentado”.